

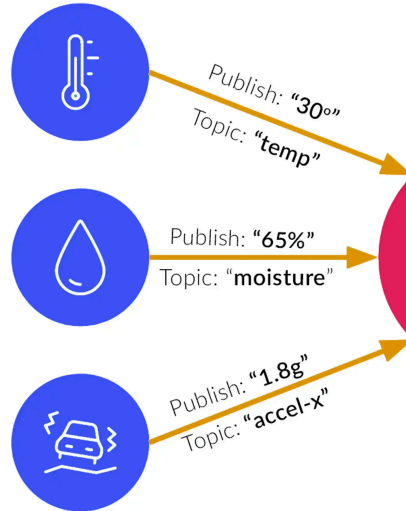
MQTT Protokoll implementálása a szimulátorba

1. MQTT
2. QoS
3. Megvalósítás
4. Az első prompt
5. QoS megvalósítása - MqttBroker.java
6. Message Tracking - MqttMessage.java
7. Timed
8. 10.000 eszközös próba - report.md
9. A teljes prompt - Copilot.md

MQTT

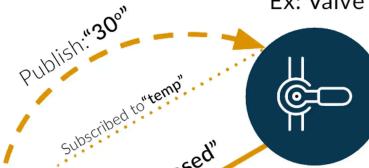
MQTT Clients

Ex: Sensors

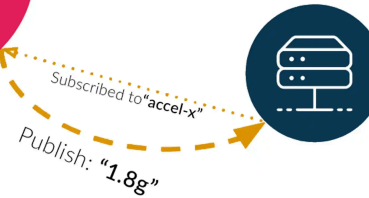


MQTT Clients

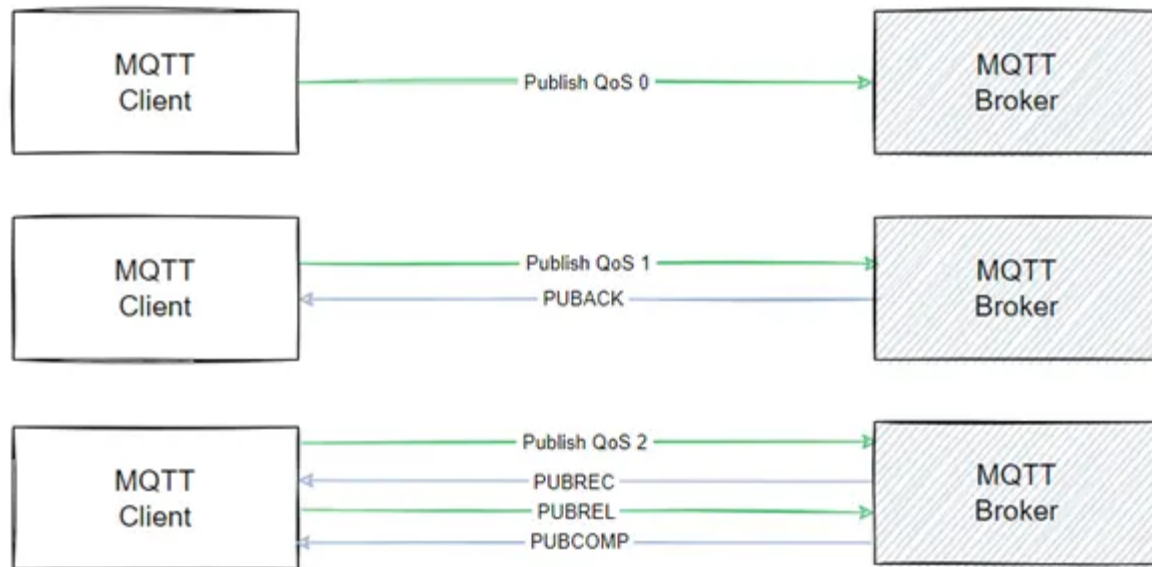
Ex: Valve



Ex: Cloud Server

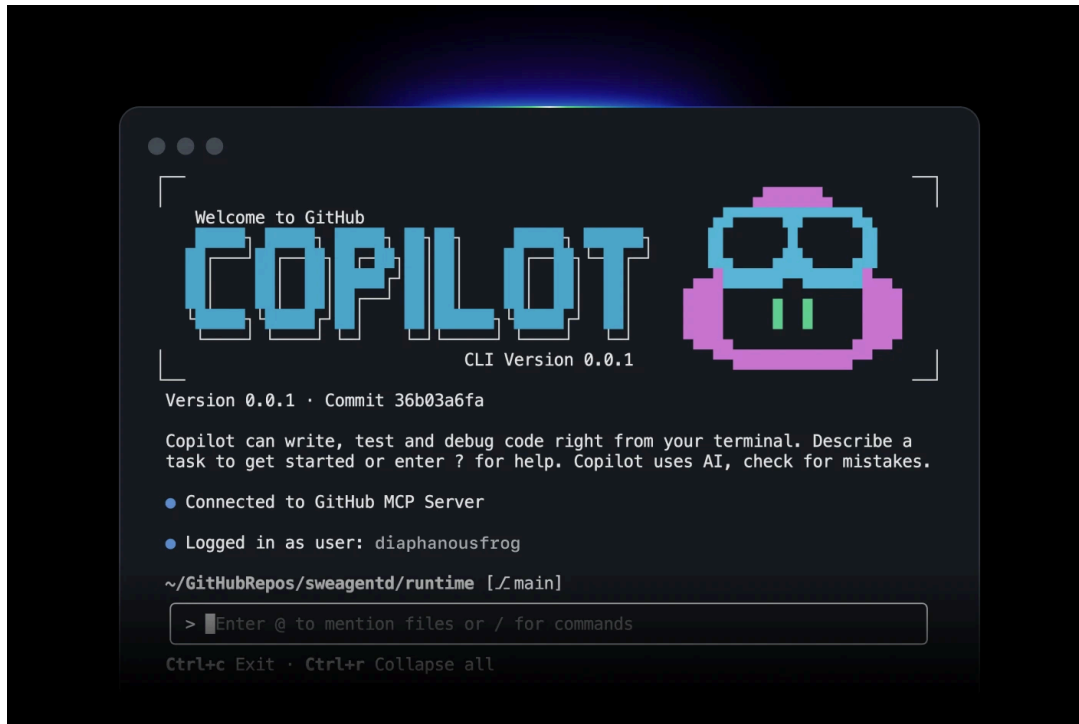


QoS



Megvalósítás

- Claude Sonnet 4.5 - Agent mód



Az első prompt

- from the PDF

Do this task:

1) Implementation of the MQTT Protocol

Building on the current functionality of the simulator, the goal is to model the operation of the MQTT protocol, which is a lightweight messaging protocol for IoT devices.

MQTT Broker: Clients (publishers and subscribers) communicate through the broker. The broker is responsible for receiving messages from publishers and routing them to the appropriate subscribers.

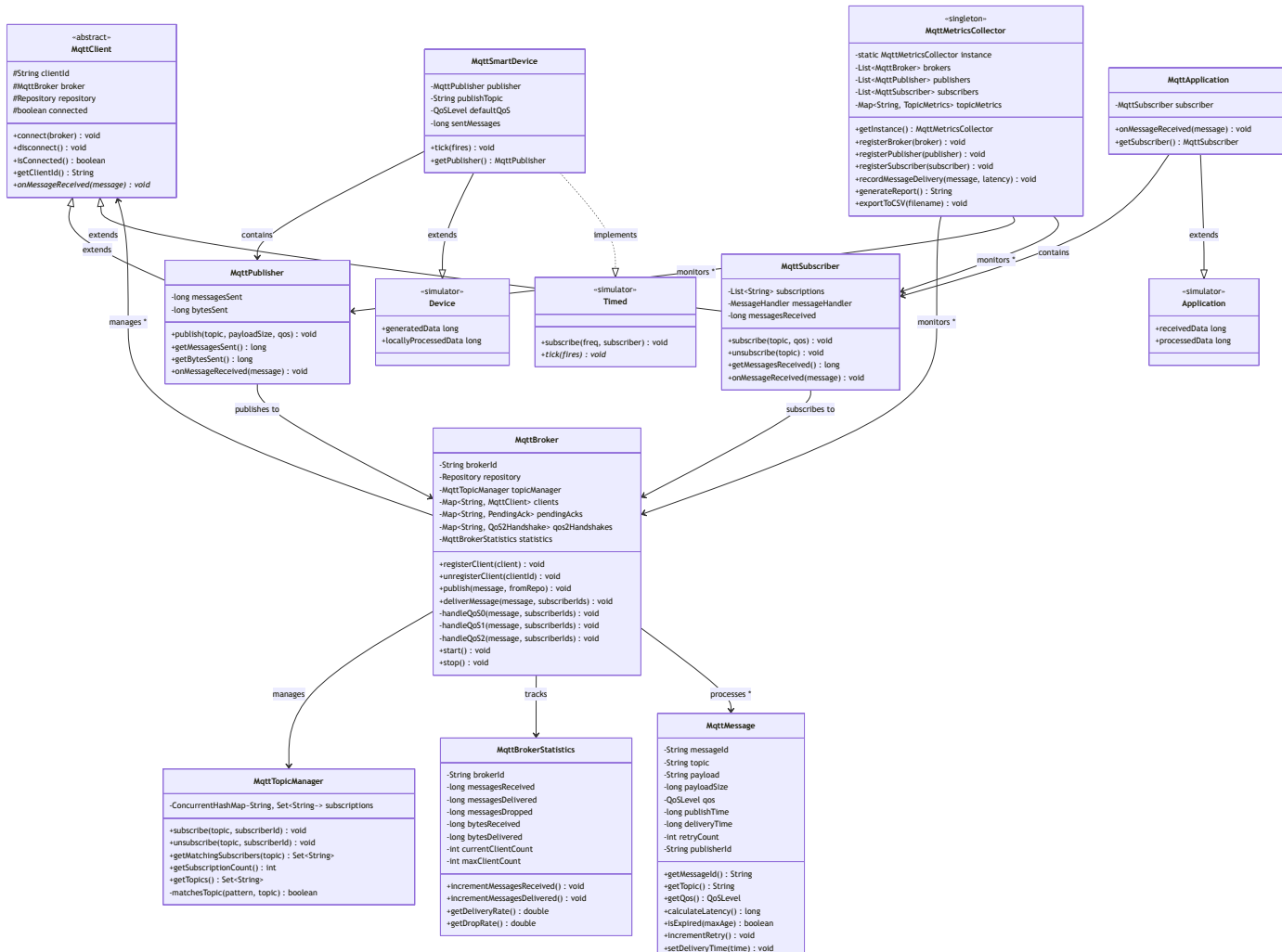
Topics: Topics are strings that define message routing. They can represent specific data types (e.g., temperature/sensor/room1) or general categories (e.g., temperature).

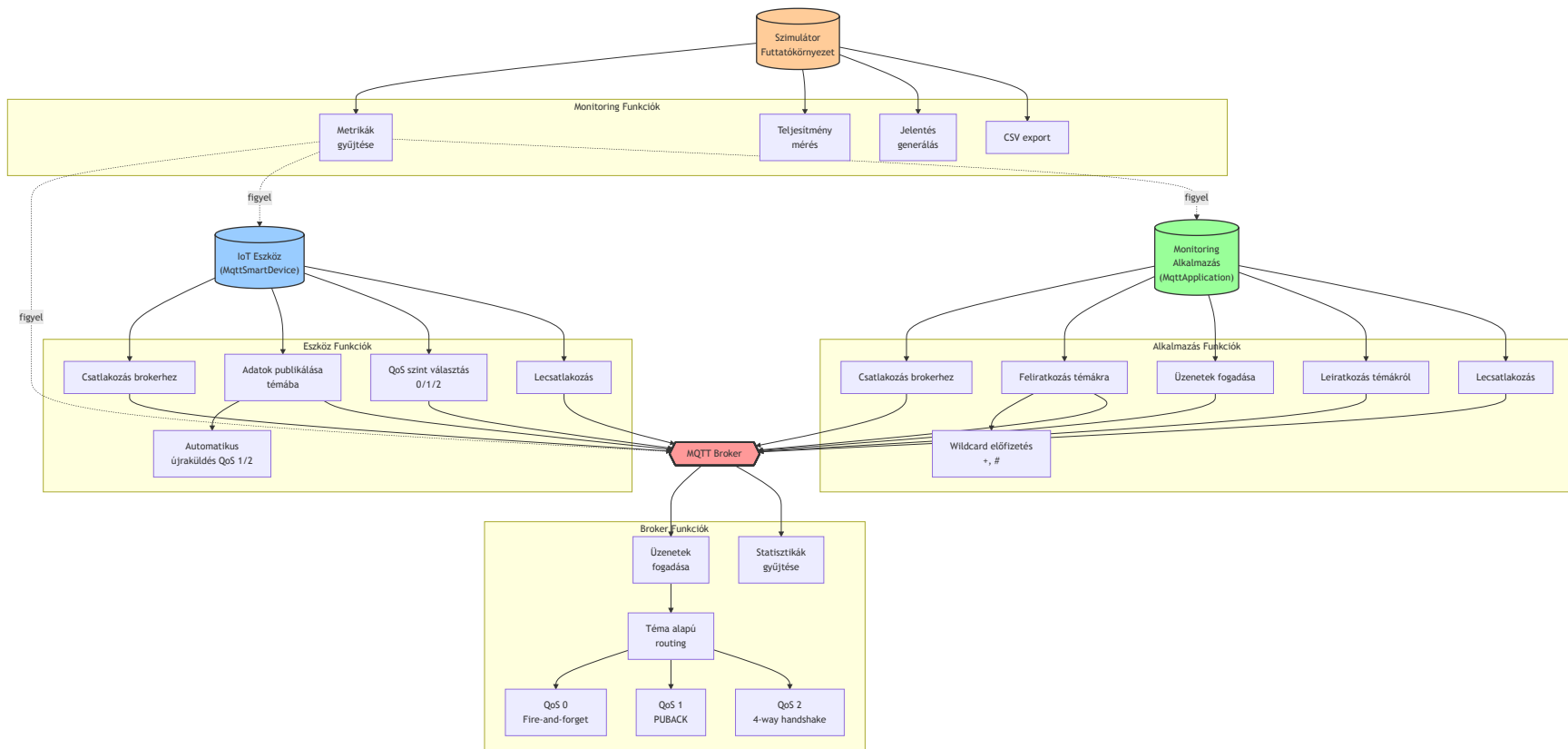
QoS Levels - These levels define how reliably messages are delivered:

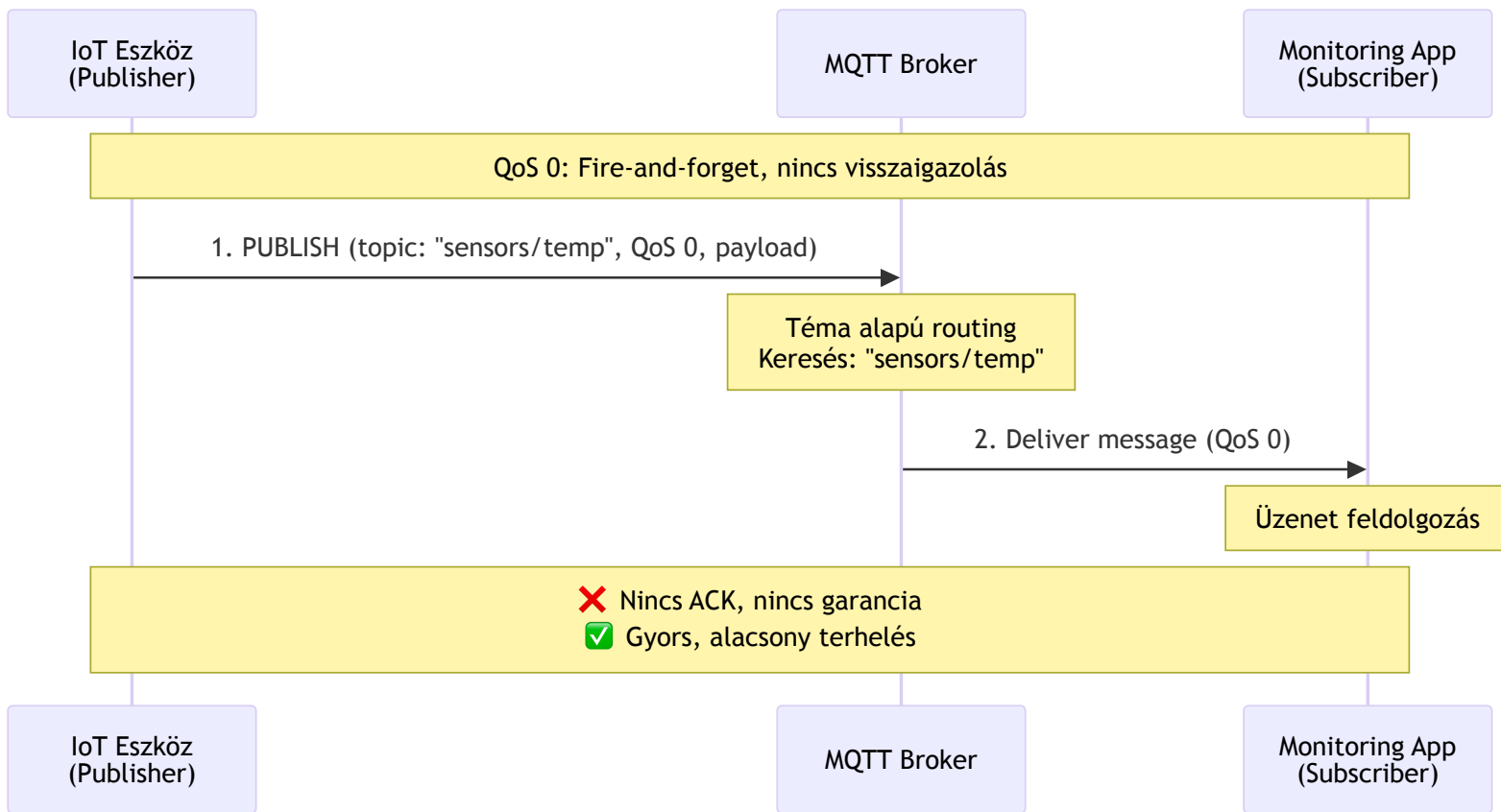
- QoS 0 ("At most once") - no acknowledgment from the broker, messages may be lost
- QoS 1 ("At least once") - the message is acknowledged but may be delivered multiple times
- QoS 2 ("Exactly once") - ensures the message is delivered only once but increases communication overhead.

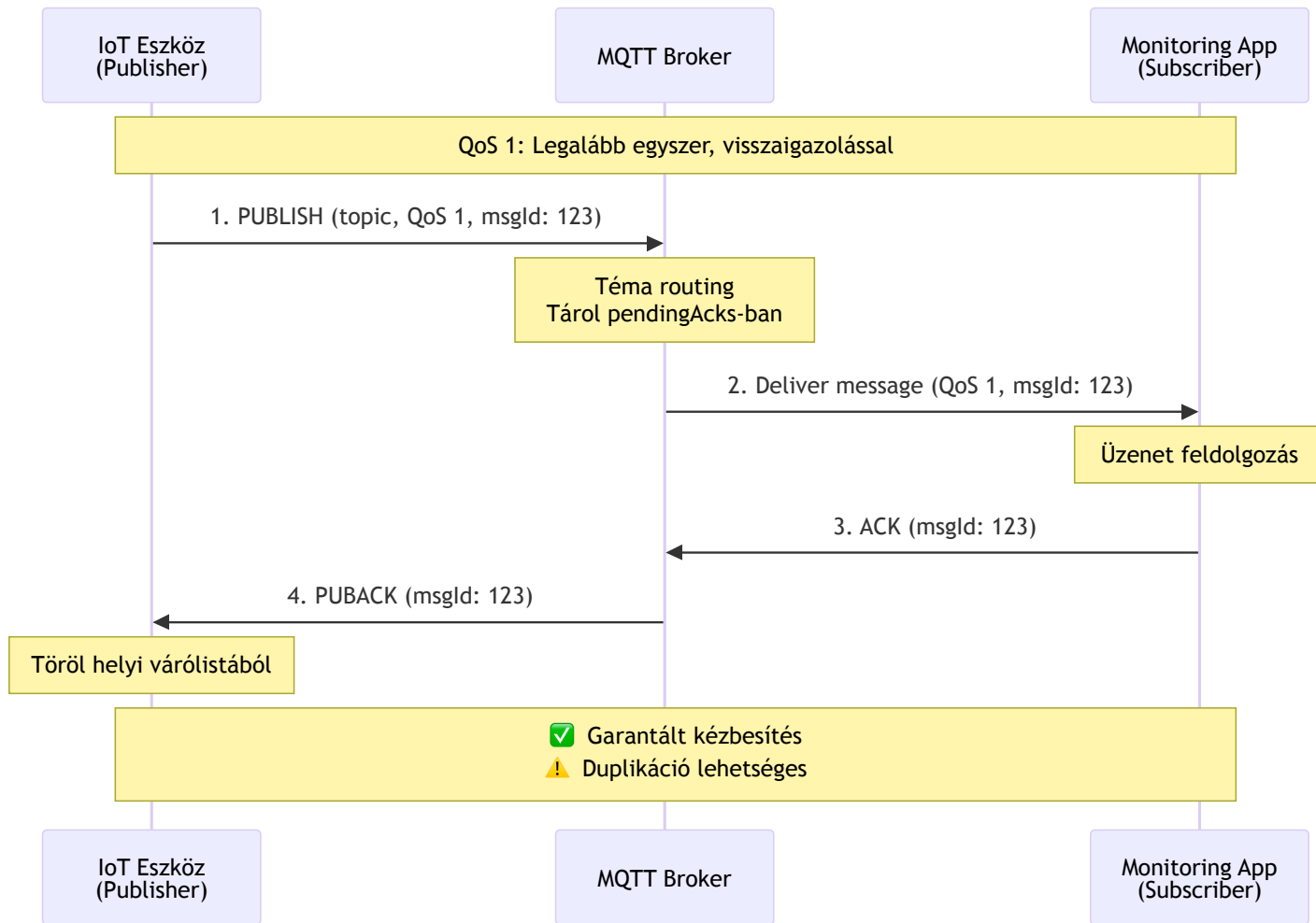
Your evaluation should be comprehensive, i.e., you should test your implementation using various configurations and inputs.

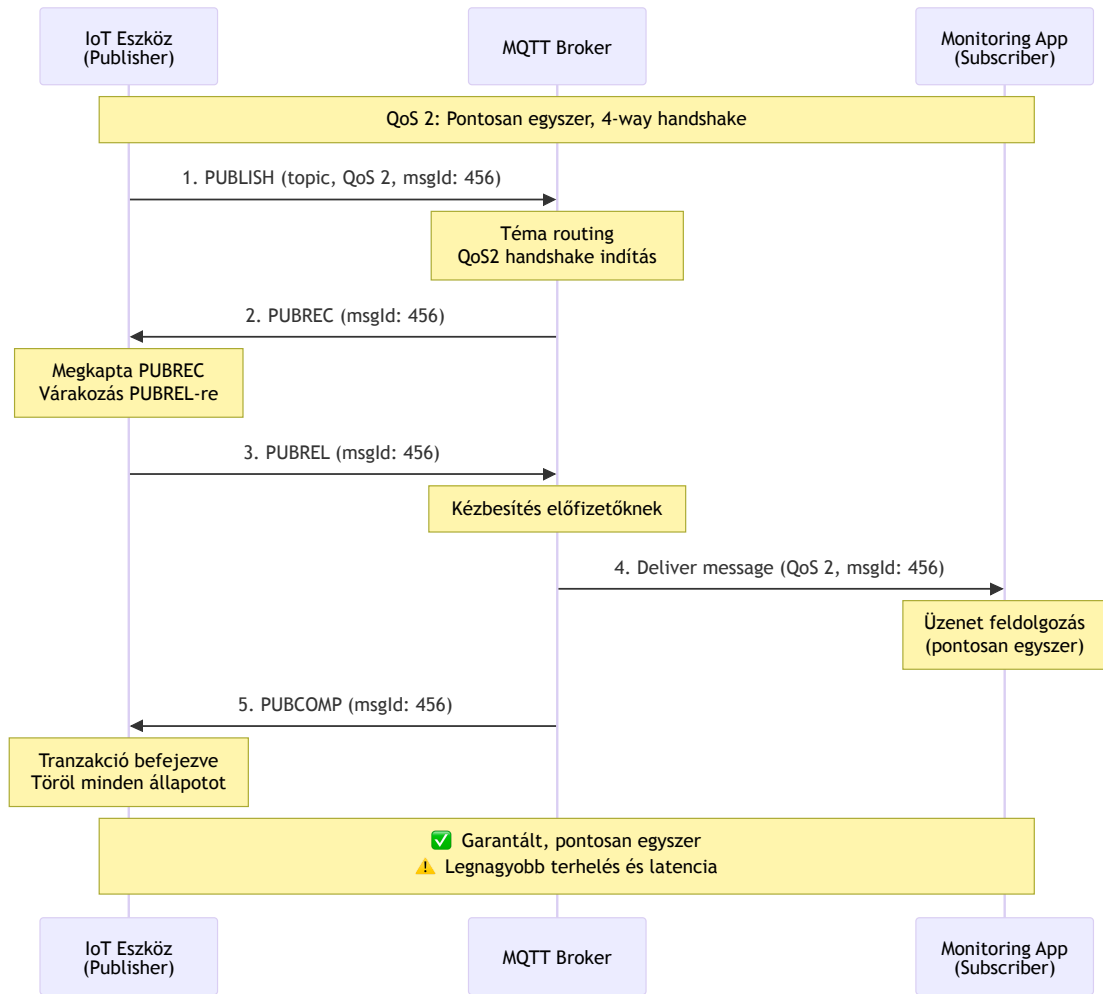
To produce comparable and reproducible results, your implementation should also include logging and measurement features.











QoS megvalósítása - MqttBroker.java

```
// simulator/src/main/java/hu/u_szeged/inf/fog/simulator/iot/mqtt/MqttBroker.java
// Lines 319-440: QoS level implementation
```

```
// Handle different QoS levels
switch (message.getQoSLevel()) {
    case AT_MOST_ONCE:
        deliverQoS0(message, subscriber);
        break;
    case AT_LEAST_ONCE:
        deliverQoS1(message, subscriber);
        break;
    case EXACTLY_ONCE:
        deliverQoS2(message, subscriber);
        break;
}
```

```
// QoS 0: Fire and forget - no acknowledgment
private void deliverQoS0(MqttMessage message, MqttClient subscriber) {
    NetworkNode.initTransfer(message.getPayloadSize(), ...,
        new ConsumptionEventAdapter() {
            public void onComplete() {
                subscriber.onMessageReceived(message);
            }
        }
    );
}
```

Message Tracking - MqttMessage.java

```
public class MqttMessage {
    // Calculates the end-to-end latency from publish to subscriber delivery.
    public long getEndToEndLatency() {
        if (subscriberReceiveTime == -1) {
            return -1;
        }
        return subscriberReceiveTime - publishTime;
    }
    // Calculates the broker processing time.
    public long getBrokerProcessingTime() {
        if (brokerReceiveTime == -1 || subscriberReceiveTime == -1) {
            return -1;
        }
        return subscriberReceiveTime - brokerReceiveTime;
    }
    @Override
    public String toString() {
        return "MqttMessage{" +
            "id=" + messageId + ", topic='" + topic + '\'' + ", payloadSize=" + payloadSize + ", qos=" + qosLevel +
            ", publisher='" + publisherId + '\'' + ", publishTime=" + publishTime + ", attempts=" + deliveryAttempts
        + " }";
    }
}
```

Timed

```
public class MqttBroker extends Timed {
    public void start() {
        if (!running) {
            running = true;
            lastActivityTime = Timed.getFireCount();
            subscribe(processingFrequency);
        }
    }
    @Override
    public void tick(long fires) {
        if (!running) return;
        if (Timed.getFireCount() - lastActivityTime > idleTimeout) {
            System.out.println("[MQTT Broker] Idle timeout - stopping broker");
            stop();
            return;
        }
        // Üzenetek feldolgozása a message queue-ból...
    }
    public void stop() { running = false; unsubscribe(); /* Leállítja a tick események fogadását*/ }
}
```

```
public class MqttPublisher extends MqttClient {
    private final Repository repository;
    private long messagesSent;
    private long bytesSent;

    public void publish(String topic,
                        long payloadSize,
                        QoSLevel qosLevel) {
        if (!connected) {
            SimLogger.logRun("Error: Not connected"); return;
        }

        MqttMessage message = new MqttMessage(topic, payloadSize, qosLevel, clientId);
        broker.publish(message, repository);

        messagesSent++; bytesSent += payloadSize;

        SimLogger.logRun("Sent message " + message.getMessageId() + " to topic '" + topic + "'");
    }
}
```

```

public class MqttSubscriber extends MqttClient {
    public void subscribe(String topicFilter,
                          QoSLevel qosLevel) {
        if (!connected) {
            SimLogger.logRun("Error: Not connected"); return;
        }

        broker.subscribe(clientId, topicFilter, qosLevel);
        subscriptions.add(topicFilter);
        SimLogger.logRun("Subscribed to '" + topicFilter + "'");
    }

    @Override
    public void onMessageReceived(
        MqttMessage message) {
        messagesReceived++;
        bytesReceived += message.getPayloadSize();

        SimLogger.logRun("Received message " + message.getMessageId() + " on topic '" + message.getTopic() +
            "' (latency: " + message.getEndToEndLatency() + "ms)");

        if (messageHandler != null) {
            messageHandler.accept(message);
        }
    }
}

```



```

public class SimpleMqttQoSExample {
    public static void main(String[] args) throws Exception {
        // ..., Create broker repository
        long brokerStorage = 10_737_418_240L; /* 10 GB */ long brokerBandwidth = 125_000; /* 1 Gbps */ Map<String, Integer> latencyMap = new HashMap<>();
        LatencyMap.put("broker-repo", 5);

        Repository brokerRepo = new Repository(brokerStorage, "broker-repo", brokerBandwidth, brokerBandwidth, brokerBandwidth);

        // Create MQTT broker with 100ms processing frequency and 5 second idle timeout
        MqttBroker broker = new MqttBroker(brokerRepo, 100, 5000); MqttMetricsCollector.getInstance().registerBroker(broker);

        // Create devices with different QoS levels
        QoSLevel[] qosLevels = {QoSLevel.AT_MOST_ONCE, QoSLevel.AT_LEAST_ONCE, QoSLevel.EXACTLY_ONCE}; String[] qosNames = {"at-most-once", "at-least-once", "exactly-once"};

        List<MqttSmartDevice> devices = new ArrayList<>(); List<MqttSubscriber> subscribers = new ArrayList<>();

        for (int i = 0; i < qosLevels.length; i++) {
            // Create device repository
            LatencyMap.put("device-repo-" + i, 5); Repository deviceRepo = new Repository(4_294_967_296L, "device-repo-" + i, 125_000, 125_000, 125_000);
            deviceRepo.setState(NetworkNode.State.RUNNING);

            PhysicalMachine devicePm = new PhysicalMachine(1, 0.001, 1_073_741_824L, deviceRepo, 1, 1, cpuTransitions);

            // Create MQTT device
            String topic = "sensors/test/device" + i;
            MqttSmartDevice device = new MqttSmartDevice(0, 10000, 500, 2000, /* start, stop, size, freq */ new StaticMqttSmartDevicePolicy());
            devices.add(device); MqttMetricsCollector.getInstance().registerPublisher(device.getPublisher());
        }
    }
}

```

10.000 eszközös próba - report.md

```
===== Simulation Completed =====
```

```
Real execution time: 77,88 seconds
```

```
Simulated time: 40.001 seconds
```

```
Time compression ratio: 0,51x
```

```
===== MQTT Simulation Metrics Report =====
```

```
=== Simulation Overview ===
```

```
Duration: 40001 ms
```

```
Brokers: 1
```

```
Publishers: 10000
```

```
Subscribers: 99
```

```
=== Message Statistics ===
```

```
Total messages sent: 359302
```

```
Total messages received: 2803538
```

```
Total bytes sent: 1796510000 (1713,29 MB)
```

```
Total bytes received: 14017690000 (13368,31 MB)
```

```
Average message latency: 100,62 ms
```

10.000 eszközös próba - report.md

```
=== QoS Level Statistics ===
```

```
QoS 0 (AT_MOST_ONCE):
```

```
  Successful deliveries: 2048071
```

```
  Failed deliveries: 0
```

```
  Success rate: 100,00%
```

```
  Average latency: 99,48 ms
```

```
QoS 1 (AT_LEAST_ONCE):
```

```
  Successful deliveries: 489755
```

```
  Failed deliveries: 0
```

```
  Success rate: 100,00%
```

```
  Average latency: 103,96 ms
```

```
QoS 2 (EXACTLY_ONCE):
```

```
  Successful deliveries: 265712
```

```
  Failed deliveries: 0
```

```
  Success rate: 100,00%
```

```
  Average latency: 103,24 ms
```

A teljes prompt - Copilot.md

```
# Laci: Do this task:
```

1) Implementation of the MQTT Protocol

Building on the current functionality of the simulator, the goal is to model the operation of the MQTT protocol, which e

MQTT Broker: Clients (publishers and subscribers) communicate through the broker. The broker is responsible for receiving

Topics: Topics are strings that define message routing. They can represent specific data types (e.g., temperature/sensor)

QoS Levels - These levels define how reliably messages are delivered:

- QoS 0 ("At most once") - no acknowledgment from the broker, messages may be lost
- QoS 1 ("At least once") - the message is acknowledged but may be delivered multiple times
- QoS 2 ("Exactly once") - ensures the message is delivered only once but increases communication overhead.

Your evaluation should be comprehensive, i.e., you should test your implementation using various configurations and inputs.

To produce comparable and reproducible results, your implementation should also include logging and measurement features

```
# Github Copilot: I'll analyze this task and create a comprehensive plan for implementing the MQTT protocol in the DISS
```

Let me first explore the current codebase to understand the existing structure:

```
Read [(file:///Users/voroslaszlo/git/DISSECT-CF-Fog/simulator/src/main/java/hu/u_szeged/inf/fog/simulator/demo/simple/
```

Searched codebase for "communication protocol network transfer IoT device message broker", 17 results